

PRODUCT REQUIREMENTS DOCUMENT

Smart Task Manager

A Python Full Stack Capstone Project — FastAPI · SQLAlchemy · SQLite · JavaScript

Version: 1.0

Date: July 26, 2026

Prepared for: Python Full Stack course — capstone teaching project

Prepared by: Digital Asset Management

1. Purpose

Smart Task Manager is a small but complete task-management web application. Its purpose is not just to "work" — it exists to give students one real, running application in which every topic taught in the Python Full Stack course shows up naturally, in context, instead of as an isolated exercise.

By the end of building it, a student will have touched: Python project structure, FastAPI, databases, request validation, error handling, and a JavaScript frontend that talks to all of it live in the browser.

2. Background

Most course exercises teach one concept at a time (a FastAPI route here, a SQLAlchemy query there). Students often struggle to see how these pieces connect into something a company would actually ship. Smart Task Manager solves that by being one continuous project, built in layers, where each new lesson plugs into the previous one.

3. Target Audience

- Students in a Python Full Stack course who already know basic Python and have been introduced to modules/packages.
- Instructors who want a single reference project to teach from, session by session.

4. Goals

1. Give students a working, professional-feeling application (not a toy demo).
2. Cover every topic in the course curriculum inside one coherent codebase.
3. Keep the code readable — a beginner should be able to open any file and understand what it does within a few minutes.
4. Demonstrate the full request journey: browser → JavaScript → HTTP → FastAPI → SQLAlchemy → SQLite → back to the browser, with no page reloads.

5. Success Criteria

- The application runs locally with two commands: one to start the backend, one to open the frontend.
- A user can create, view, search, filter, update, and delete tasks, and see live statistics — all without refreshing the page.
- Every file in the project maps to at least one topic in the curriculum (see Section 9).
- The accompanying documents (Design Document, Core Requirements, Task Breakdown, Teaching Guide) let an instructor teach the project step by step without reverse-engineering the code first.

6. Scope

In Scope

Task CRUD (create, read, update, delete) · Search by title/description · Filter by status and priority · Statistics endpoint (counts by status and priority) · Input validation and friendly error messages · A clean, responsive browser UI built with plain HTML/CSS/JavaScript · SQLite database via SQLAlchemy ORM

Out of Scope (flagged as future extensions)

User accounts / authentication (JWT) · Multi-user permissions · Deployment to the cloud / Docker · Real-time updates (WebSockets) · Mobile app version

These are intentionally left out so the project stays teachable in one course module — this is a design choice, not an oversight, and is called out directly to students.

7. Users & Use Cases

Primary persona — "The Learner": a student following along in class, typing code into their own copy of the project.

Typical use cases: add a new task with title/description/priority/status/deadline; search for a task by keyword; filter down to "High priority, Pending" tasks; mark a task "Completed" by editing it; delete a task; glance at the statistics panel.

8. Features (User Stories)

#	User Story	Priority
1	As a user, I can add a new task with title, description, priority, status, and deadline.	Must have
2	As a user, I can see all my tasks as cards on a dashboard.	Must have
3	As a user, I can edit an existing task's details.	Must have
4	As a user, I can delete a task I no longer need.	Must have
5	As a user, I can search tasks by keyword.	Must have
6	As a user, I can filter tasks by status and/or priority.	Must have
7	As a user, I can see a summary of task counts (pending, completed, high priority).	Must have
8	As a user, I get a clear message if something goes wrong.	Must have
9	As a developer, invalid input is rejected automatically with a clear error.	Must have
10	As an instructor, I can point to any file and explain, in one sentence, what it's responsible for.	Must have

9. Curriculum Coverage Map

Course Module	Where It Shows Up in This Project
Modules and Packages	The whole <code>app/</code> folder — code is split into <code>database.py</code> , <code>models.py</code> , <code>schemas.py</code> , <code>crud.py</code> , <code>routers/</code> instead of one giant file.
Scripting Best Practices	Naming conventions, constants in <code>utils.py</code> , logging, <code>requirements.txt</code> , docstrings on every file.
FastAPI Introduction	<code>app = FastAPI()</code> in <code>main.py</code> , the very first route.
Advanced FastAPI	<code>APIRouter</code> , <code>Depends()</code> for the database session, response models, status codes, tags.
API Project	The entire backend — <code>POST/GET/PUT/DELETE /tasks</code> , <code>GET /tasks/stats</code> .
API Testing	Testing every endpoint using the built-in Swagger UI (<code>/docs</code>) and Postman.

JavaScript Essentials	<code>app.js</code> — variables, functions, objects, arrays, loops, DOM manipulation, event listeners, JSON.
Asynchronous JavaScript	Every <code>async function</code> in <code>app.js</code> that uses <code>await fetch(...)</code> to talk to the backend without reloading the page.
NodeJS Essentials	Explained conceptually and used practically through the VS Code "Live Server" extension (itself a small Node/npm tool) that serves the frontend files.
API Fundamentals	The full client → HTTP → server → database → JSON → client round trip, traced explicitly in the Design Document.
Request Validation	Pydantic schemas in <code>schemas.py</code> (<code>Literal</code> fields, <code>min_length</code> , required vs optional fields).
Exception Handling	Custom <code>TaskNotFoundException</code> , 404/400/500 handling in <code>exceptions.py</code> and <code>main.py</code> .
Database Integration	SQLAlchemy ORM (<code>models.py</code> , <code>database.py</code> , <code>crud.py</code>) reading and writing to SQLite.

10. Assumptions & Constraints

- Students have Python 3.10+ and VS Code installed.
- No cloud hosting is required — everything runs on `localhost` .
- The frontend and backend run as two separate local servers (frontend via VS Code "Live Server", backend via `uvicorn`), connected over HTTP with CORS enabled — this deliberately mirrors how real frontend/backend teams work, even at small scale.

11. Risks

Risk	Mitigation
Students get lost switching between many files	The Teaching Guide walks through the project in a fixed, numbered order, one file at a time.
CORS errors confuse beginners	The Design Document explains what CORS is and why the error happens, in plain language, before students hit it.
Async JavaScript is a conceptually hard topic	The Concept Explanations document dedicates a full section to it with a plain-language analogy.

12. Appendix — Non-Goals Explicitly Called Out to Students

To keep expectations realistic, students should be told directly that authentication, deployment, and multi-user support are **intentionally** left for a future module — this is a design choice, not an oversight.